

Projet compilation

Gabriel VALDÉS-ALONZO

3 mai 2025

1 Introduction

Ce projet consiste en la création d'un compilateur écrit en langage C, conçu pour traiter des expressions régulières et générer un programme capable de vérifier si des mots appartiennent au langage défini par ces expressions. On implemente ce projet en étapes: on crée un arbre syntaxique associé à l'expression. Cet arbre est utilisé pour construire un automate non déterministe (NFA), qui est ensuite converti en un automate déterministe (DFA). Le DFA est ensuite minimisé pour obtenir un automate minimal unique qui génère un fichier C contenant un programme qui peut être compilé séparément. Ce programme permet de vérifier si un mot donné appartient au langage défini par l'expression régulière initiale ou pas.

2 Structure du code

Dans cette section on parle de la structure du code, en partant pour parler de comme le projet est organisé au niveau de la conception jusqu'à au niveau des fonctions et philosophie.

2.1 Structure du projet

Le code pour le compilateur est écrit en C. On suit une structure traditionnelle avec une fonction `main` comme point de départ et des fonctions pour améliorer la compréhension. De cette manière, on utilise des fichiers `.c` pour écrire le code, et des fichiers `.h` pour déclarer les fonctions et ajouter les bibliothèques nécessaires.

Le répertoire du projet est structuré de manière d'avoir une répartition logique des différents composants. La première division est au niveau des répertoires dans le projet, qui sont organisés de la manière suivante :

Code

```
|_ build/  
|_ doc/  
|_ files/  
|_ out/  
|_ src/
```

où les différents répertoires ont chacun sa fonction:

- **build** : répertoire réservé pour tous les sorties du fichier **Makefile**. Cela concerne tous les fichiers intermédiaires `.o` et aussi l'exécutable après la compilation.
- **doc** : répertoire contenant la documentation et les fichiers `tex` utilisés pour le générer.
- **files** : répertoire réservé pour les fichiers d'entrée du code, particulièrement le fichier `.txt` avec l'expression régulière à analyser.
- **out** : répertoire réservé pour tous les sorties du code. Dans ce projet en particulier les sorties sont les fichiers `.dot` qui représentent les différents objets du code (les automates et l'arbre syntaxique), les images générés par *graphviz* et le fichier `.c` généré à partir de l'automate minimale.

- **src** : répertoire réservé pour le code source, c'est-à-dire tous les fichiers `.c` et `.h` avec les fonctions et codes implémentent les automates et le code de sortie.

Finalement, dans le répertoire **src** on a un fichier **Makefile**, qui est là pour simplifier la compilation du code. Le fichier peut être appelé avec la commande shell **make**. Si l'on veut refaire la compilation on peut appeler avant la commande **make clean**, commande qui appelle simplement **rm -rf ../build**.

2.2 Structure des fichiers

Le point d'entrée du code est la fonction **src/main.c**, qui reçoit des arguments et délègue aux autres fonctions. Comme l'algorithme commence avec un arbre syntaxique, suivi des différents automates avant d'exporter le programme de sortie, on divise les fonctions par sujet :

- **src/main.c** : Code principal. Cette fonction appelle les autres fonctions dans cette liste.
- **src/f_tree.c** : Algorithme pour créer l'arbre syntaxique. La fonction utilise un algorithme où on lit l'expression régulière de gauche à droite, lettre par lettre et ajoute les feuilles de l'arbre avec les opérations nécessaires. Elle utilise des fonctions auxiliaires définies dans **f_aux_tree.c**.
- **src/f_nfa.c** : Algorithme qui prend l'arbre syntaxique et génère le NFA. Par chaque feuille de l'arbre elle crée un sous-automate qui est combiné selon les opérations correspondantes. Elle utilise les fonctions auxiliaires définies dans **f_aux_automata.c**.
- **src/f_dfa.c** : Algorithme qui crée l'automate déterministe à partir du NFA. Elle utilise les fonctions auxiliaires définies dans **f_aux_automata.c** pour faire le parcours du NFA et obtenir les groupes d'états combinés.
- **src/f_dfa_min.c** : Algorithme qui minimise le DFA et obtient l'automate minimal unique. Les fonctions auxiliaires sont définies dans **f_aux_automata.c**.
- **src/f_export.c** : Fonction qui prend l'automate minimal et exporte un fichier `.c` de sortie qui peut être compilé séparément. Cette programme peut prendre un mot et vérifier s'il appartient au langage défini par l'expression régulière initiale.
- **src/f_aux_*** : Fonctions auxiliaires. Les fonctions sont détaillées dans les sections 3.3 et 3.4.
- **src/aux_structs.h** : Fichier où on déclare les structures qui définissent les objets à utiliser, détaillé dans la section 3.1.

3 Documentation des fonctions

Dans cette section on détaille les fonctions utilisées et l'action qu'elles font dans le programme. Les noms des fonctions (en anglais) sont descriptifs de l'action qu'elles exécutent, donc il n'est pas nécessaire de faire une explication très détaillée, sauf s'il y a un algorithme utilisé. De toute façon, le code a des commentaires pour donner du contexte au code et d'expliquer les fonctionnalités basiques des algorithmes.

3.1 aux_structs

Le code est organisé autour des objets pour simplifier les relations entre les différentes parties. Comme le langage C est pas orienté aux objets, on utilise des **structs** pour avoir les fonctionnalités d'encapsulation de données. Comme ces objets sont pas définies de la même manière que dans autres langages comme C++, il faut être attentif à l'allocation de mémoire et l'utilisation des pointeurs. Les objets utilisés sont:

Arbre syntaxique et ses feuilles

L'arbre syntaxique est le première objet, que facilite l'encapsulation des variables associés à l'arbre. Le code du **struct** est

```
1  typedef struct SyntaxTree {
2      int num_leaves;           // Nombre de feuilles dans l'arbre
3      TreeNode *root;          // Pointeur vers la racine de l'arbre
4      TreeNode **leaves;       // Tableau de pointeurs vers les feuilles.
5      int num_unique_chars;     // Quantite de caracteres uniques.
6      char* unique_chars;      // Caracteres uniques de l'expression.
7  } SyntaxTree;
```

où on utilise un système récursif pour les feuilles. On garde un pointeur vers la racine, la quantité des feuilles, un tableau de pointeurs vers toutes les feuilles et des infos sur les caractères dans l'expression régulière. Chaque feuilles est aussi un objet :

```
1  typedef struct TreeNode {
2      char value;               // Caractere de l'arbre
3      int index;                // Index du noeud dans l'arbre
4      struct TreeNode *parent;  // Pointeur vers le parent
5      struct TreeNode *left_child; // Pointeur vers le fils gauche
6      struct TreeNode *right_child; // Pointeur vers fils droite ou NULL
7  } TreeNode;
```

où on garde la valeur, l'indice de la feuille et des pointeurs vers le père et les enfants.

Automates et ses parts

Logiquement, c'est aussi avantageux de garder l'automate dans une structure de données fixe. L'automate est définie comme un **struct** de la manière suivante :

```
1  typedef struct Automaton {
2      int num_states;           // Nombre d'etats de l'automate
3      int num_transitions;      // Nombre de transitions de l'automate
4      int num_unique_chars;     // Quantite de caracteres uniques.
5      char* unique_chars;       // Caracteres uniques de l'expression.
6      AState **states;          // Liste des etats de l'automate
7      ATransition **transitions; // Liste des transitions de l'automate
8  } Automaton;
```

où on garde la quantité d'états et transitions, la quantité des caractères uniques et les caractères, et aussi un tableau des états et transitions. Les états et transitions sont aussi des objets définies comme

```

1  typedef struct AState {
2      int index;                // Indice de l'etat
3      int is_start;             // Indicateur d'etat de depart
4      int is_final;             // Indicateur d'etat final
5      int is_deleted;           // Indicateur d'etat ignore.
6      int num_transitions;      // Nombre de transitions sortantes
7      int num_in_trans;         // Nombre de transitions sortantes
8      ATransition **transitions; // Pointeur vers les etats suivantes
9      ATransition **in_trans;   // Pointeur vers les etats suivantes
10     NodeSet *states_set;       // Indices des etats contenus
11 } AState;

```

```

1  typedef struct ATransition {
2      char symbol;              // Symbole de la transition
3      struct AState *from;      // Etat de depart de la transition
4      struct AState *to;        // Etat d'arrivee de la transition
5  } ATransition;

```

où pour l'état on garde l'indice, des variables boolean pour identifier si sont des états initiaux, finaux ou supprimés (C n'a pas de boolean natif, donc on utilise un `int`), la quantité des transitions entrantes et sortantes, des tableaux des pointeurs vers ces transitions et un set d'états, réservé pour aider dans l'algorithme des automates déterministes. Le set d'états est défini comme

```

1  typedef struct NodeSet {
2      int *node_array;          // Liste de nodes
3      int array_size;           // Taille de la liste
4  } NodeSet;

```

où on garde simplement la taille du set et les indices des états concernés.

3.2 Main functions

- `SyntaxTree *create_syntax_tree(char *expression) :`

Fonction qui crée un arbre syntaxique à partir d'une expression régulière. La fonction lit l'expression de gauche à droite et ajoute les feuilles à l'arbre en considérant les opérations correspondents. L'algorithme est décrit ci-dessous.

```

1      var: pile de feuilles de l arbre
2      var: pile auxiliaires des operateurs
3
4      for (chaque caractere dans l expression)
5          | if (caractere == '(')
6              | | if (caractere precedent == lettre|'*'|')')
7                  | | ajouter concatenation a la pile operateur;
8                  | | ajouter parenthese a la pile feuille;
9              | if (caractere == ')')
10                 | | on depile les operateurs jusqu a fermer parenthese;
11                 | if (caractere == '|')
12                     | | on ajoute disjonction a la pile operateur;
13                 | if (caractere == '*')
14                     | | on ajoute l etoile a la pile feuille;
15                 | | si la pile a concatenation, on bouge vers les feuilles;
16                 | if (caractere == lettre)

```

```

17 | | if (caractere precedent == ')')
18 | | | on depile dernier operateur (concat ou union);
19 | | on ajoute la lettre;
20 | | if (caractere precedent == '*' | lettre)
21 | | | on ajoute concatenation a l arbre
22
23 if (il reste des elements dans pile operateur)
24 | on processe l operateur;
25 | on ajoute operateur a l arbre;
26
27 return arbre;

```

- **Automaton** *create_nfa_from_syntax_tree(**SyntaxTree** *tree) :

Crée l'automat non déterministe à partir de l'arbre. Elle prend chaque feuille et l'ajoute comme un sous-automate, qui est combiné avec les autres. L'algorithme est :

```

1   entre : etat initial sous-automate
2   entre : etat final sous-automate
3
4   for (chaque feuille de l arbre)
5   | if (feuille == ')')
6   | | continue;
7   | if (feuille == '.')
8   | | on cree transition vide entre etat initial 2 et final 1;
9   | | on ajoute transitions et etats au nfa;
10  | | on ajoute transitions aux etats;
11  | if (feuille == '|')
12  | | on cree deux etats;
13  | | on cree 4 transitions vides;
14  | | on ajoute transitions et etats au nfa;
15  | | on ajoute transitions aux etats;
16  | if (feuille == '*')
17  | | on cree deux etats;
18  | | on cree 4 transitions vides;
19  | | on ajoute transitions et etats au nfa;
20  | | on ajoute transitions aux etats;
21  | if (feuille == lettre)
22  | | on cree deux etats;
23  | | on cree une transition avec la lettre;
24  | | on ajoute transitions et etats au nfa;
25  | | on ajoute transitions aux etats;

```

- **Automaton** *create_dfa_from_nfa(**Automaton** *nfa) :

Fonction qui prend un automate non déterministe et crée un automate déterministe associé en regroupant des états. L'algorithme est :

```

1   on cherche etat de debut;
2   on cherche epsilon-cloture;
3
4   do
5   | for (chaque etat)
6   | | for (chaque lettre)
7   | | | on cherche tous les chemins avec la lettre;

```

```

8   while (le nombre d etats incremente)
9
10  on cherche et marque les etats finaux;

```

- **Automaton** *create_minimal_dfa(**Automaton** *dfa) :

Fonction que minimise l'automat déterministe, en donnant l'automate minimal unique.

```

1   on cherche les etats acceptants;
2   on cherche les non-acceptants;
3
4   do
5   | for (chaque groupe)
6   | | for (chaque etat dans le groupe)
7   | | | for (chaque lettre dans l etat)
8   | | | | on cherche si les etats ont differents debuts et departs;
9   | if (il y a des etats differents)
10  | | on cree un nouvel set;
11  | | if (set n existe pas)
12  | | | on l ajoute au dfa minimal;
13  while (nombre de sets incremente)
14
15  for (chaque etat du dfa minimal)
16  | on marque les etats de depart ou finaux;
17
18  on cree tableau des transitions;
19  for (chaque transition du dfa minimal)
20  | on ajoute au tableau;
21
22  if (transition est nouvelle)
23  | on ajoute au dfa minimal;
24
25  for (chaque etat du dfa minimal)
26  | if (etat deconecte)
27  | | on marque comme supprime;

```

- **void** export_automate_code(**Automaton** *dfa, **const char** *expression, **const char** *filename) :

Fonction pour exporter l'automate minimal. Elle écrit un code C associé a cet automate, qui peut être compilé séparément pour analyser des mots. La fonction écrit un tableau de transitions et des boucles pour lire le mot lettre par lettre et vérifier s'il appartient ou pas a le langage décrit par l'expression régulière.

3.3 f_aux_automata

Tous les fonctions auxiliaires dans les algorithmes des automates sont dans ce fichier. Il y a 4 types.

3.3.1 Fonctions pour initialiser des objets

Toutes ces fonctions initialisent des objets, c'est-à-dire, elles allouent la mémoire pour les attributs dedans et donne des valeurs par défaut, car on peut pas garantir que C le fait. Partic-

ulièrement il faut être attentif avec les pointeurs et tableaux, où il faut s'assurer que les pointeurs son nulles.

- `void initialize_nfa(Automaton *nfa, SyntaxTree *tree) :`
Cette fonction aussi compte la quantité des états et transitions avant l'algorithme, donc le tableau et déjà alloué avant commencer.
- `void initialize_dfa(Automaton *dfa, Automaton *nfa)`
- `void initialize_state(AState *state, int index, int is_start, int is_final)`
- `void initialize_dfa_state(AState *state, int index, int is_start, int is_final, NodeSet *node_set)`
- `void initialize_transition(ATransition *transition, AState *from, AState *to, char symbol)`

3.3.2 Fonctions pour modifier les automates

Ces fonctions modifient les automates. On les utilisé pour ajouter des données aux automates (états et transitions), ce que l'on fait en modifiant les compteurs et en réallouent la mémoire pour chaque tableau.

- `void add_transition_to_state(ATransition *transition, AState *state_from, AState *state_to)`
- `void add_transition_to_nfa(Automaton *automat, ATransition *transition, int counter)`
- `void add_transition_to_dfa(Automaton *dfa, ATransition *transition)`
- `void add_state_to_nfa(Automaton *automat, AState* state, int counter)`
- `void add_state_to_dfa(Automaton *nfa, AState *state)`
- `void merge_states_automaton(Automaton *automat) :`
Cette fonction est utilisé pour combiner des états dans la concatenation. Normalement on combine deux DFA avec une état commun pour les deux, mais dans notre algorithme on utilise un état intermédiaire avec une transition vide (marqué par un arrobas). Cette fonction prendre l'état et le supprime de l'automate, en réorientent les transitions vers les état correctes.

3.3.3 Fonctions pour faire des actions

- `void follow_path_from_single_state(NodeSet *node_set, AState *initial_state, char value) :`
Fonction qui prend un état de départ dans le NFA, un caractère et cherche tous les états accesibles. Chaque état atteint est ajouté dans un group des états. Les transitions vides sont considérés aussi, mais seulement s'il sont pas la première transition. Un pseudo code associé est donné ci-dessous.


```

1   for (tous les transitions dans un etat)
2   | if (transition vide dans un groupe non vide OU
3   |     transition du caractere cherche)
4   | | if (etat non ajoute dans le groupe)
5   | | | ajouter etat;
6   | | | rappeler la fonction avec le nouvel etat;

```

- `void follow_path_from_group_state(Automaton *dfa, Automaton *nfa, AState *initial_state, char value) :`

Fonction qui prend un état de départ dans le DFA (donc un set d'états) et cherche tous les chemins possibles associés a un caractère donné. Elle appelle la fonction `follow_path_from_single_state` recursivement.

```

1   for (tous les etats NFA dans un etat DFA)
2   | follow_path_from_single_state;
3   |
4   | if (group d etats est vide)
5   | | return;;
6   | |
7   | | if (groupe existe OU groupe est contenu dans un autre)
8   | | | ajouter une transition;
9   | | | return;
10  | |
11  | | on cree un nouvel group;
12  | | on ajoute les transitions;
13  | | return;

```

- `void check_final_states(Automaton *dfa, Automaton *nfa) :`

Vérifie quels états du NFA sont finaux et les marque comme finaux dans le DFA.

- `void remove_states_from_set(NodeSet *original_set, NodeSet *new_set) :`

Vérifie si les états dans `new_set` sont dans `original_set`. Si ça ce le cas, elle supprime l'état dans le set original et réalloue la mémoire pour le tableau réduit.

- `int is_state_in_node_set(int index, NodeSet *node_set) :`

Vérifie si l'état avec indice `index` est contenu dans le set `node_set`. Elle retour 1 si ce le cas et 0 si non.

- `int compare_node_sets(NodeSet *set1, NodeSet *set2) :`

Vérifie si le set1 et le set2 sont égaux. Elle retour 1 si ce le cas, 0 si non.

- `int contained_node_set(NodeSet *set1, NodeSet *set2) :`

Vérifie si le set2 est contenu dans le set 1. Elle retour 1 si le set est contenu et 0 si non.

3.3.4 Fonctions pour exporter les données

- `void export_automaton_to_graphviz(const char *filename, Automaton *automat) :`

Cette fonction prendre l'automat (DFA ou NFA) et l'exporte dans un format `.dot` qui peut être utilisé pour *graphviz* pour dessiner le graphe.

3.4 f_aux_tree

3.4.1 Fonctions pour initialiser des objets

- `initialize_leaf(TreeNode *leaf, char value, int index)` :
Fonction qui initialise les feuilles de l'arbre. Elle marque les pointeurs comme nulles pour sécuriser la mémoire.

3.4.2 Fonctions pour faire des actions

- `int verify_syntax_tree(char *expression)` :
Fonction qui lit l'expression et vérifie si elle est correcte. Elle vérifie si tous les parenthèses sont fermés, si les opérateurs comme la disjonction sont appliqués correctement. La fonction aussi compte le nombre de feuilles que l'arbre aura, en ajoutant aussi les concatenations implicites.
- `find_unique_alphanumerics(const char* input, char* output, int* out_count)` :
Implemente un algorithme pour identifier les caractères utilisés dans l'expression. Pour chaque lettre de l'expression vérifie si le caractère a été lu et l'ajoute à un tableau. Elle retourne un `string` (`const char*`) avec tous les caractères alphanumériques uniques.

3.4.3 Fonctions pour exporter les données

Les fonctions suivantes exportent l'arbre dans le format `.dot`. Comme l'arbre est définie d'une manière récursive, on define deux fonction, une qui exporte les feuilles et autre qui exporte l'arbre, qui fait une appel récursif à la fonction du feuille.

- `export_node(FILE *f, TreeNode *node)`
- `export_tree_to_graphviz(const char *filename, SyntaxTree *tree)`